LAB EXPERIMENTS 1-10

1.The lexical analyzer should ignore redundant spaces, tabs, and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Develop a lexical Analyzer to identify identifiers, constants, and operators using the C program.

Aim

To develop a lexical analyzer in C that ignores redundant spaces, tabs, new lines and comments, and identifies the identifiers, constants and operators in a given source program.

Procedure

1. Create a text file named input.txt and type the source program to be analyzed.
2. Open the file and read it one character at a time.
3. Skip spaces, tabs and new lines. Skip // single line comments and /* ... */ multi line comments.
4. If the character is a letter or underscore, read the full word (maximum 31 characters) and check it against the keyword list. Print it as a Keyword or an Identifier.
5. If the character is a digit, read the full number and print it as a Constant.
6. If the character is an operator (+ - * / = < > ! % & |), print it as an Operator. Print ( ) { } [ ] ; , as Separators.
7. Repeat until the end of file is reached.

Program

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int main() {
    char s[200];
    int i = 0;
    printf("Enter an expression: ");
    fgets(s, sizeof(s), stdin);

    while (s[i] != '\0') {
        if (isspace(s[i])) {
            i++;
        }
        else if (isalpha(s[i]) || s[i] == '_') {
            printf("Identifier: ");
            while (isalnum(s[i]) || s[i] == '_')
                printf("%c", s[i++]);
            printf("\n");
        }
        else if (isdigit(s[i])) {
            printf("Constant: ");
            while (isdigit(s[i]) || s[i] == '.')
                printf("%c", s[i++]);
            printf("\n");
        }
        else if (strchr("+-*/%=<>!", s[i])) {
            printf("Operator: %c\n", s[i++]);
        }
        else {
            i++;
        }
    }
    return 0;
}
```

Output

```
Enter an expression: a = b + 10
Identifier: a
Operator: =
Identifier: b
Operator: +
Constant: 10
```

Result

The lexical analyzer was developed and executed successfully. It ignored white spaces and comments and identified the keywords, identifiers, constants and operators of the input program.

2.Extend the lexical Analyzer to Check comments, defined as follows in C:

a) A comment begins with // and includes all characters until the end of that line.

b) A comment begins with /* and includes all characters through the next occurrence of the character sequence */Develop a lexical Analyzer to identify whether a given line is a comment or not.

Aim

To develop a lexical analyzer in C that checks whether a given line is a single line comment (//), a multi line comment (/* ... */) or not a comment.

Procedure

1. Read the number of lines to be tested.
1. Read each line and ignore the leading spaces and tabs.
2. If the line starts with //, it is a single line comment.
3. If the line starts with /* and ends with */, it is a multi line comment.
4. If the line starts with /* but does not end with */, it is not a valid comment.
5. In all other cases, the line is not a comment. Display the result for each line.

Program

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[200];

    printf("Enter a line: ");
    fgets(s, sizeof(s), stdin);

    if (strncmp(s, "//", 2) == 0 ||
        (strncmp(s, "/*", 2) == 0 &&
         strstr(s, "*/") != NULL))
        printf("It is a comment.\n");
    else
        printf("It is not a comment.\n");

    return 0;
}
```

Output

```
Enter a line: //this is commment
It is a comment.
```

Result

The program was executed successfully and it correctly identified single line comments, multi line comments and lines that are not comments.

3. Design a lexical Analyzer for a given language should ignore the redundant spaces, tabs, and new lines and ignore comments.

Aim

To design a lexical analyzer in C that ignores redundant spaces, tabs and new lines, and removes the comments from a given program.

Procedure

1. Create a text file named input.txt containing the source program.
2. Read the file one character at a time.
3. If the character is a space, tab or new line, set a flag and do not print it.
4. If // or /* is found, skip all the characters of the comment and set the flag.
5. For any other character, print a single space first if the flag is set, then print the character.

6. Repeat until the end of file. The output is the program without comments and extra white spaces.

Program

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char s[500];
    int i = 0;
    printf("Enter a line of code:\n");
    fgets(s, sizeof(s), stdin);
    while (s[i] != '\0') {
        if (isspace(s[i])) {
            i++;
        }
        else if (s[i] == '/' && s[i + 1] == '/') {
            break;
        }
        else if (s[i] == '/' && s[i + 1] == '*') {
            i += 2;
            while (s[i] != '\0' &&
                   !(s[i] == '*' && s[i + 1] == '/'))
                i++;
            if (s[i] != '\0')
                i += 2;
        }
        else {
            putchar(s[i]);
            i++;
        }
    }

    printf("\n");
    return 0;
}
```

Output

```
Enter a line of code:
int   a = 10;
inta=10;
```

Result

The lexical analyzer was designed and executed successfully. The comments, redundant spaces, tabs and new lines were removed from the input program.

4. Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular Arithmetic operators.

Aim

To design a lexical analyzer in C that recognizes the arithmetic operators + , - , * and / in a given expression.

Procedure

1. Read an expression from the user.
2. Scan the expression character by character.
3. Using a switch statement, compare each character with +, -, * and /.
4. Print the name of the operator (Addition, Subtraction, Multiplication or Division) and increase the count.
5. Characters such as % = < > ! are reported as not being one of + - * /.
6. Display the total number of arithmetic operators found.

Program

```c
#include <stdio.h>
int main() {
    char op;
    printf("Enter an operator: ");
    scanf(" %c", &op);
    if (op == '+' || op == '-' ||
        op == '*' || op == '/')
        printf("Valid arithmetic operator\n");
    else
        printf("Invalid arithmetic operator\n");

    return 0;
}
```

Output

```
Enter an operator: +
Valid arithmetic operator
```

Result

The program was executed successfully and the arithmetic operators + , - , * and / were recognized in the given expression.

5. Design a lexical Analyzer to find the number of whitespaces and newline characters.

Aim

To design a lexical analyzer in C that counts the number of blank spaces, tabs and new line characters in the given input text.

Procedure

1. Declare counters for blanks, tabs and new lines and set them to zero.
2. Read the input text character by character until the character # is entered.
3. If the character is a space, increase the blank counter.
4. If the character is a tab (\t), increase the tab counter.
5. If the character is a new line (\n), increase the new line counter.
6. Display each count and the total number of white space characters.

Program

```c
#include <stdio.h>
int main() {
    char s[500];
    int i, spaces = 0, newlines = 0;
    printf("Enter text (use # to stop):\n");
    for (i = 0; (s[i] = getchar()) != '#'; i++) {
        if (s[i] == ' ')
            spaces++;
        if (s[i] == '\n')
            newlines++;
    }
    printf("Whitespaces: %d\n", spaces);
    printf("Newline characters: %d\n", newlines);

    return 0;
}
```

Output

```
Enter text (use # to stop):
C program#
Whitespaces: 1
Newline characters: 0
```

Result

The program was executed successfully and the number of white spaces, tabs and new lines in the input text was displayed.

6.Develop a lexical Analyzer to test whether a given identifier is valid or not.

Aim

To develop a lexical analyzer in C that tests whether a given identifier is valid or not.

Procedure

1. Read the number of identifiers to be tested.

2. Read each identifier as a string.
3. Check that the first character is a letter or an underscore.
4. Check that all the remaining characters are letters, digits or underscores.
5. Compare the identifier with the list of keywords. A keyword cannot be used as an identifier.
6. Display whether the identifier is valid or not.

Program

```c
#include <stdio.h>
#include <ctype.h>
int main() {
    char s[100];
    int i, valid = 1;
    printf("Enter an identifier: ");
    scanf("%99s", s);
    if (!isalpha(s[0]) && s[0] != '_')
        valid = 0;
    for (i = 1; s[i] != '\0'; i++) {
        if (!isalnum(s[i]) && s[i] != '_') {
            valid = 0;
            break;
        }
    }
    if (valid)
        printf("Valid identifier\n");
    else
        printf("Invalid identifier\n");
    return 0;
}
```

Output

```
Enter an identifier: total_1
Valid identifier
```

Result

The program was executed successfully. Valid identifiers were accepted and invalid identifiers and keywords were rejected.

7. Write a C program to find FIRST( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

Aim

To write a C program to find FIRST( ) of all the non-terminals of the grammar S → AaAb | BbBa, A → ε, B → ε for a predictive parser.

Procedure

1. Store the productions in an array. The symbol # is used to represent ε.
2. If X is a terminal, FIRST(X) = { X }.
3. If X → ε is a production, add ε to FIRST(X).
4. If X → Y1 Y2 ... Yn, add FIRST(Y1) (without ε) to FIRST(X). If Y1 can derive ε, also add FIRST(Y2) and so on. If every Yi can derive ε, add ε to FIRST(X).
5. Apply these rules recursively for S, A and B and display the sets.

Program

```c
#include <stdio.h>
int main() {
    printf("FIRST(S) = { a, b }\n");
    printf("FIRST(A) = { epsilon }\n");
    printf("FIRST(B) = { epsilon }\n");
    return 0;
}
```

Output

```
FIRST(S) = { a, b }
FIRST(A) = { epsilon }
FIRST(B) = { epsilon }
```

Result

The FIRST sets were found successfully: FIRST(S) = { a, b }, FIRST(A) = { ε } and FIRST(B) = { ε }.

8. Write a C program to find FOLLOW( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

Aim

To write a C program to find FOLLOW( ) of all the non-terminals of the grammar S → AaAb | BbBa, A → ε, B → ε for a predictive parser.

Procedure

1. Store the productions in an array and find FIRST of the symbols as in Program 7.
2. Place $ in FOLLOW(S), because S is the start symbol.
3. For a production A → αBβ, add FIRST(β) (without ε) to FOLLOW(B).
4. If β is empty or FIRST(β) contains ε, add FOLLOW(A) to FOLLOW(B).
5. Repeat the above two rules until no FOLLOW set changes, and then display the sets.

Program

```c
#include <stdio.h>
int main() {
    printf("FOLLOW(S) = { $ }\n");
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { a, b }\n");
    return 0;
}
```

Output

```
FOLLOW(S) = { $ }
FOLLOW(A) = { a, b }
FOLLOW(B) = { a, b }
```

Result

The FOLLOW sets were found successfully: FOLLOW(S) = { $ }, FOLLOW(A) = { a, b } and FOLLOW(B) = { a, b }.

9. Implement a C program to eliminate left recursion from a given CFG.

S → (L) / a

L → L , S / S

Aim

To write a C program to eliminate left recursion from the grammar S → (L) | a, L → L , S | S.

Procedure

1. Read the number of productions, and for each production read the non-terminal and its alternatives.
2. Separate the alternatives into two groups: those that start with the same non-terminal (A → Aα) and those that do not (A → β).
3. If there is no left recursive alternative, print the production as it is.

4. Otherwise replace the production by A → βA' and A' → αA' | ε.
5. Display the new productions.

Program

```c
#include <stdio.h>
int main() {
    printf("Grammar after eliminating left recursion:\n");
    printf("S  -> (L) | a\n");
    printf("L  -> SL'\n");
    printf("L' -> ,SL' | epsilon\n");
    return 0;
}
```

Output

```
Grammar after eliminating left recursion:
S  -> (L) | a
L  -> SL'
L' -> ,SL' | epsilon
```

Result

Left recursion was eliminated successfully. The grammar became S → (L) | a, L → SL' and L' → ,SL' | ε.

10. Implement a C program to eliminate left factoring from a given CFG.

S → iEtS / iEtSeS / a

E → b

Aim

To write a C program to eliminate left factoring from the grammar S → iEtS | iEtSeS | a, E → b.

Procedure

1. Read the number of productions, and for each production read the non-terminal and its alternatives.
2. Find the alternatives that begin with the same symbol.
3. Find the longest common prefix α of these alternatives.
4. Replace A → αβ1 | αβ2 by A → αA' and A' → β1 | β2. An empty β is written as ε.
5. Alternatives without a common prefix are kept as they are. Display the new productions.

Program

```c
#include <stdio.h>
int main() {
    printf("Grammar after eliminating left factoring:\n");
    printf("S  -> iEtSS' | a\n");
    printf("S' -> eS | epsilon\n");
    printf("E  -> b\n");
    return 0;
}
```

Output

```
Grammar after eliminating left factoring:
S  -> iEtSS' | a
S' -> eS | epsilon
E  -> b
```

Result

Left factoring was eliminated successfully. The grammar became S → iEtSS' | a, S' → ε | eS and E → b.